

南開大學

信息隐藏技术课程实验报告

实验四：媒体文件格式剖析内容



学 院 密码与网络空间安全学院
专 业 信息安全、法学双学位班
学 号 2313815
姓 名 段俊宇
班 级 信息安全、法学双学位班

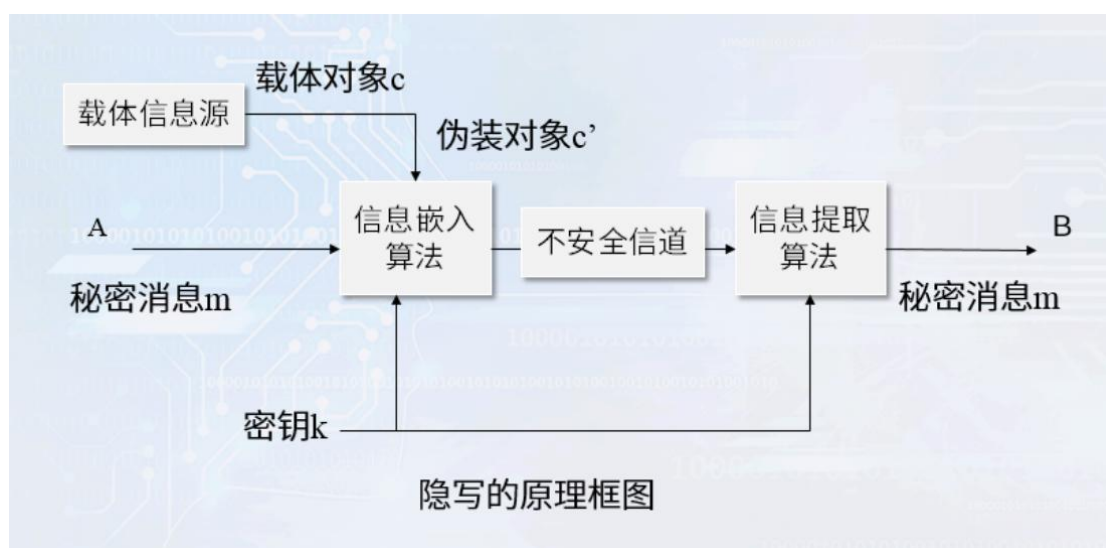
一、实验要求

- 1、 任选一种图像文件，进行格式剖析。
- 2、 针对该类型的文件，讨论可能的隐藏位置和隐藏方法。
- 3、 实现秘密信息的隐藏和提取。

二、实验原理

隐写的基本概念：

- 载体对象：A 打算秘密传递一些信息给 B, A 需要从一个随机消息源中随机选取一个无关紧要的消息 c , 当这个消息公开传递时, 不会引起怀疑, 称这个消息 c 为载体对象。
- 伪装对象：把需要秘密传递的信息 m 隐藏到载体对象 c 中, 此时, 载体对象 c 就变为伪装对象 c' 。
- 伪装密钥：秘密信息的嵌入过程需要密钥, 此密钥称为伪装密钥。



实现隐写的基本要求：

- 载体对象是正常的, 不会引起怀疑。
- 伪装对象与载体对象无法区分, 无论从感观还是从计算机的分析。
- 不可视通信的安全性取决于第三方有没有能力将载体对象和伪装对象区别开。
- 对伪装对象的正常处理, 不应破坏隐藏的信息。

三、实验环境

四、实验过程

1. 图像文件格式剖析

如下所示，我将使用注释的方式对代码进行解释。

```
1. % 分析 BMP 文件格式的函数
2. function analyze_bmp()
3.     % 读取 BMP 文件
4.     file_path = '../Picture/1.bmp';
5.     fid = fopen(file_path, 'r');
6.     if fid == -1
7.         error('无法打开文件');
8.     end
9.
10. % 读取文件头信息, 14 字节
11. file_header = fread(fid, 14, 'uint8=>uint8');
12.
13. % 解析文件头
14. bfType = char(file_header(1:2)); % 文件类型
15. bfSize = typecast(file_header(3:6), 'uint32'); % 文件大小
16. bfReserved1 = typecast(file_header(7:8), 'uint16'); % 保留字段 1
17. bfReserved2 = typecast(file_header(9:10), 'uint16'); % 保留字段 2
18. bfOffBits = typecast(file_header(11:14), 'uint32'); % 像素数据偏移量
19.
20. % 读取信息头, 40 字节
21. info_header = fread(fid, 40, 'uint8=>uint8');
22.
23. % 解析信息头
24. biSize = typecast(info_header(1:4), 'uint32'); % 信息头大小
25. biWidth = typecast(info_header(5:8), 'int32'); % 图像宽度
26. biHeight = typecast(info_header(9:12), 'int32'); % 图像高度
27. biPlanes = typecast(info_header(13:14), 'uint16'); % 色彩平面数
28. biBitCount = typecast(info_header(15:16), 'uint16'); % 每像素位数
29. biCompression = typecast(info_header(17:20), 'uint32'); % 压缩类型
30. biSizeImage = typecast(info_header(21:24), 'uint32'); % 图像大小
31. biXPelsPerMeter = typecast(info_header(25:28), 'int32'); % 水平分辨率
32. biYPelsPerMeter = typecast(info_header(29:32), 'int32'); % 垂直分辨率
33. biClrUsed = typecast(info_header(33:36), 'uint32'); % 使用的颜色数
34. biClrImportant = typecast(info_header(37:40), 'uint32'); % 重要颜色数
35.
36. % 关闭文件
37. fclose(fid);
```

```

38.
39. % 输出结果
40. fprintf('=== BMP 文件格式分析 ===\n');
41. fprintf('文件类型: %s\n', bfType);
42. fprintf('文件大小: %d 字节\n', bfSize);
43. fprintf('保留字段 1: %d\n', bfReserved1);
44. fprintf('保留字段 2: %d\n', bfReserved2);
45. fprintf('像素数据偏移量: %d 字节\n', bfOffBits);
46. fprintf('信息头大小: %d 字节\n', biSize);
47. fprintf('图像宽度: %d 像素\n', biWidth);
48. fprintf('图像高度: %d 像素\n', biHeight);
49. fprintf('色彩平面数: %d\n', biPlanes);
50. fprintf('每像素位数: %d\n', biBitCount);
51. fprintf('压缩类型: %d\n', biCompression);
52. fprintf('图像大小: %d 字节\n', biSizeImage);
53. fprintf('水平分辨率: %d\n', biXPelsPerMeter);
54. fprintf('垂直分辨率: %d\n', biYPelsPerMeter);
55. fprintf('使用的颜色数: %d\n', biClrUsed);
56. fprintf('重要颜色数: %d\n', biClrImportant);

```

我选择 bmp 格式的图像文件进行分析，它由文件头、信息头、颜色表以及像素阵列构成，当位深度为 24 时，没有颜色表。实验用的图片如下所示：



上面的代码分别对文件头和信息头进行了读取，结果如下所示：

1. 文件类型: BM
2. 文件大小: 11771898 字节
3. 保留字段 1: 0
4. 保留字段 2: 0
5. 像素数据偏移量: 138 字节
6. 信息头大小: 124 字节
7. 图像宽度: 2548 像素
8. 图像高度: 1155 像素

- 9. 色彩平面数: 1
- 10. 每像素位数: 32
- 11. 压缩类型: 3
- 12. 图像大小: 11771760 字节
- 13. 水平分辨率: 3779
- 14. 垂直分辨率: 3779
- 15. 使用的颜色数: 0
- 16. 重要颜色数: 0

先对文件头进行处理, 文件类型标识 BM 代表 BMP 格式文件; 文件头第 3-6 个字节表示文件大小, 该图片大小约为 11.2MB; 第 7 到 10 字节表示保留字段, 它们通常为 0; 像素数据偏移量是数据的起始位置, 输出表明数据相对于位图的位置从第 138 字节开始。

之后是信息头的处理, 图像宽度和高度分别在信息头第 5-8 字节和第 9-12 字节, 从这里可以看出图片的宽度为 2548 像素, 高度为 1155 像素; 色彩平面数表示图像数据中颜色平面的数量, 该字段必须为 1; 像素位数为 32 位, 代表增强真彩色; 该图片的压缩类型为 3, 代表 BI_BITFIELDS, 表示每个像素值由指定的掩码决定; 信息头同样存储了位图大小, 与文件头相同数值; 信息头第 25-28 字节和第 29-32 字节分别代表水平分辨率和垂直分辨率, 这里都是 3779; 使用的颜色数为 0 表示使用全部颜色, 由于当前图片的像素位是 32, 因此没有颜色表, 直接使用 RGB 值; 重要颜色数为 0 表示所有颜色都重要, 不需要优先加载某些颜色。

2. 可能的隐藏位置和隐藏方法

基于该图像的信息隐藏有如下几种位置:

- 像素数据的最低有效位: 由于人类对颜色的细微变化不敏感, 并且像素值的最低 1-2 位被修改基本不影响图像质量, 因此可以将消息隐藏在这类位置。
- 文件头的保留字段: 通常来说文件头的保留字段为 0, 可以利用这一点在这里隐藏少量信息。
- 信息头的分辨率、使用颜色数和重要颜色数: 分辨率可以存储一部分信息, 但是值需要在合理区间, 或者将分辨率编码为秘密的一部分; 使用颜色数和重要颜色数可以存储部分信息, 但容量极其有限。
- 文件间隙: 在各个部分的间隙可以隐藏部分数据, 不会改变图像质

量，但是使用 Binary Ninja 之类的工具打开图片时会被检测到。
隐藏的方法主要有下面几种：

- LSB 隐写：将秘密信息的每一位嵌入到像素的最低有效位，适用于 24 位或 32 位真彩色图像。
- 调色板隐写：由于索引色图像存在颜色表，因此可以利用这一特点对颜色表中的颜色值进行 LSB 修改，或者用相似颜色替换，并将该颜色编码到消息中。
- 文件结构隐写：利用图像中的冗余空间，将信息嵌入其中实现隐写。
- 变换域隐写：在图像的 DCT 或 DWT 变换域中嵌入信息，该方法具有更好的抗检测性。

3. 随机 LSB 隐写

如下所示是隐藏函数，我将使用注释的方式对代码进行解释。

```
1. % 信息隐藏函数
2. function hide_message(input_image, secret_message, output_image, secret_key,
position_file)
3.     img = imread(input_image);
4.     [h, w, c] = size(img);
5.
6.     % 将字符串转为 UTF8 字节，兼容中文
7.     message_bytes = unicode2native(secret_message, 'UTF-8');
8.     message_length = length(message_bytes);
9.     length_bits = dec2bin(message_length, 32) - '0';
10.    message_bits = reshape(dec2bin(message_bytes, 8).' - '0', 1, []);
11.    data_bits = [length_bits message_bits];
12.
13.    % 图像容量
14.    capacity = h * w * c;
15.    if length(data_bits) > capacity
16.        error('图像容量不足');
17.    end
18.
19.    % 转为一维
20.    img_vec = img(:);
21.    total_pixels = length(img_vec);
22.
23.    % 使用密钥生成随机位置
24.    rng(secret_key);
25.    all_positions = randperm(total_pixels);
```

```

26.     positions = all_positions(1:length(data_bits));
27.
28.     % 保存位置信息
29.     save(position_file, 'positions');
30.
31.     % LSB 嵌入到随机位置
32.     for i = 1:length(data_bits)
33.         img_vec(positions(i)) = bitset(img_vec(positions(i)), 1, data_bits(i));
34.     end
35.
36.     % 恢复图像
37.     stego_img = reshape(img_vec, h, w, c);
38.     imwrite(stego_img, output_image);
39.
40.     % 计算 PSNR
41.     [mse, psnr] = calculate_psnr(img, stego_img);
42.     fprintf('隐写完成\n');
43.     fprintf('PSNR: %.2f dB\n', psnr);
44. end

```

这部分代码首先将秘密转换为二进制编码，接着检查图片当前的容量是否满足隐写的要求。为了增强抗检测性，我使用秘钥生成随机数种子，依赖种子生成随机位置并保存，这样提取时能够获取精确位置。这部分代码还计算了处理后图像的 PSNR，代码如下：

```

1. % PSNR 计算
2. function [mse, psnr] = calculate_psnr(original, stego)
3.     original = double(original);
4.     stego = double(stego);
5.     mse = mean((original(:) - stego(:)).^2);
6.     if mse == 0
7.         psnr = Inf;
8.     else
9.         psnr = 10 * log10(255^2 / mse);
10.    end
11. end

```

经过隐藏处理后的图片如下所示：



这张图片和原始图片从肉眼来看几乎没有差别，说明图像隐写成功。

如下所示是提取函数，我将使用注释的方式对代码进行解释。

```
1. % 信息提取函数
2. function message = extract_message(stego_image, position_file)
3.     fprintf('开始提取信息...\n');
4.
5.     % 加载位置信息
6.     load(position_file, 'positions');
7.
8.     img = imread(stego_image);
9.     img_vec = img(:);
10.
11.     % 读取 32bit 长度
12.     length_bits = zeros(1, 32);
13.     for i = 1:32
14.         length_bits(i) = bitget(img_vec(positions(i)), 1);
15.     end
16.     message_length = bin2dec(char(length_bits + '0'));
17.
18.     % 读取消息 bit
19.     total_bits = message_length * 8;
20.     message_bits = zeros(1, total_bits);
21.     for i = 1:total_bits
22.         message_bits(i) = bitget(img_vec(positions(i+32)), 1);
23.     end
24.
25.     % 将 bit 转换为 byte
26.     message_bytes = zeros(1, message_length);
27.     for i = 1:message_length
28.         start_idx = (i-1) * 8 + 1;
```



```

29.         end_idx = i * 8;
30.         byte = message_bits(start_idx:end_idx);
31.         message_bytes(i) = bin2dec(char(byte + '0'));
32.     end
33.
34.     % 将 UTF8 转换为字符串
35.     message = native2unicode(uint8(message_bytes), 'UTF-8');
36.     fprintf('提取完成\n');
37. end
38.

```

这部分代码首先加载位置信息，然后按照对应的位置提取隐藏的秘密，通过编码转换最终实现秘密恢复。下面是测试代码：

```

1. % 输入参数
2. input_image = '../Picture/1.bmp';
3. output_image = '../Picture/stego_image.bmp';
4. position_file = '../Picture/positions.mat'; % 保存位置信息
5. secret_message = '这是一条秘密信息! Hello World!';
6. secret_key = 2026;
7.
8. % 隐藏信息
9. hide_message(input_image, secret_message, output_image, secret_key, position_file);
10. % 提取信息
11. extracted = extract_message(output_image, position_file);
12.
13. fprintf('原始信息: %s\n', secret_message);
14. fprintf('提取信息: %s\n', extracted);
15. if strcmp(secret_message, extracted)
16.     fprintf('✓ 验证成功: 信息完全一致\n');
17. else
18.     fprintf('✗ 验证失败\n');
19. end

```

测试后的输出如下所示，说明成功实现秘密的隐藏和提取，并且实现了隐写的基本要求！

```

隐写完成
PSNR: 96.04 dB
开始提取信息...
提取完成
原始信息: 这是一条秘密信息! Hello World!
提取信息: 这是一条秘密信息! Hello World!
✓ 验证成功: 信息完全一致
>>

```

4. DWT 变换域隐写

如下所示是小组成员完成的 DWT 变换域隐藏函数和提取函数。

```
1. % 本图片背景存在大量纯黑区域。在这些区域嵌入信息后，通过逆变换重构回像素值时，若结果为负数，
   会被强制截断为 0
2. % 这种数值截断会导致提取时 DWT 系数漂移回原始状态，从而造成严重的提取错误
3. % 故将 start_idx 移至 10000，将信息嵌入到图像中部纹理丰富的区域
4. clc; clear; close all;
5. % 1. 读取原图
6. cover_path = '../Picture/1.bmp';
7. % 检查文件是否存在
8. % if ~exist(cover_path, 'file')
9. %     error('未找到原图，请确保 "独属于我的蒙娜丽莎.png" 在当前目录下。');
10. % end
11. % 加载图像数据
12. img = imread(cover_path);
13. % 强制去除 Alpha 通道，防止透明度干扰 RGB 像素值
14. if size(img, 3) == 4
15.     img = img(:,:,1:3);
16. end
17. img_work = img;
18. B = double(img_work(:,:,3));
19.
20. % 2. 准备秘密信息
21. secret_msg = '2313815 段俊宇';
22. msg_bytes = unicode2native(secret_msg, 'utf-8');
23. msg_bin_mat = dec2bin(msg_bytes, 8) - '0';
24. msg_bin = reshape(msg_bin_mat.', 1, []);
25. msg_len = length(msg_bin);
26. len_bin = dec2bin(msg_len, 32) - '0';
27. watermark = [len_bin, msg_bin];
28.
29. % 3. 进行一级二维离散小波变换
30. [cA, cH, cV, cD] = dwt2(B, 'haar');
31. cH_vec = cH(:);
32.
33. % 4. 嵌入
34. delta = 80;
35. % 避开背景死区，从图像中部开始嵌入
36. start_idx = 10000;
37.
38. % 遍历待嵌入的二进制位
39. for i = 1:length(watermark)
40.     % 计算当前位在分量向量中的索引
```

```

41.     idx = start_idx + i;
42.     % 获取当前的 DWT 系数值
43.     c = cH_vec(idx);
44.     % 获取当前待嵌入的位
45.     bit = watermark(i);
46.     % 根据待嵌入位进行量化处理
47.     if bit == 0
48.         % 嵌入 0: 量化到 delta 的偶数倍
49.         cH_vec(idx) = round(c / delta) * delta;
50.     else
51.         % 嵌入 1: 量化到 delta 的奇数倍偏移
52.         cH_vec(idx) = round((c - delta/2) / delta) * delta + delta/2;
53.     end
54. end
55. cH_emb = reshape(cH_vec, size(cH));
56.
57. % 5. 逆离散小波变换重构图像通道
58. B_emb = idwt2(cA, cH_emb, cV, cD, 'haar');
59. sz = size(B);
60. % 修正因 DWT 变换可能产生的尺寸偏差
61. B_emb = B_emb(1:sz(1), 1:sz(2));
62. B_emb(B_emb < 0) = 0;
63. B_emb(B_emb > 255) = 255;
64. img_stego = img_work;
65. img_stego(:, :, 3) = uint8(B_emb);
66.
67. % 6. 保存含密图像
68. stego_path = '../Picture/stego_1.bmp';
69. imwrite(img_stego, stego_path);
70.
71. % 打印完成提示
72. disp('===== DWT 隐写嵌入成功 =====');
73. disp(['秘密信息: ', secret_msg]);
74. disp(['含密图像已保存为: ', stego_path]);

```

```

1. clc; clear; close all;
2.
3. % 1. 读取含密图像
4. stego_path = '../Picture/stego_1.bmp';
5. % if ~exist(stego_path, 'file')
6. %     error('未找到含密图像, 请先运行 dwt_exp.m 生成含密图片。');
7. % end
8. img_stego = imread(stego_path);
9. B_stego = double(img_stego(:, :, 3));
10.

```

```

11. % 2. 进行一级二维离散小波变换
12. [cA, cH, cV, cD] = dwt2(B_stego, 'haar');
13. % 提取水平细节分量并展平为向量
14. cH_vec = cH(:);
15.
16. % 设置量化步长
17. delta = 80;
18. % 设置起始索引
19. start_idx = 10000;
20.
21. % 3. 提取前 32 位长度信息
22. len_bin = zeros(1, 32);
23. for i = 1:32
24.     idx = start_idx + i;
25.     c = cH_vec(idx);
26.     d0 = abs(c - round(c / delta) * delta);
27.     d1 = abs(c - (round((c - delta/2) / delta) * delta + delta/2));
28.     % 根据欧氏距离最小原则进行位判决
29.     if d0 < d1
30.         % 距离 d0 更近, 判定为位 0
31.         len_bin(i) = 0;
32.     else
33.         % 距离 d1 更近, 判定为位 1
34.         len_bin(i) = 1;
35.     end
36. end
37.
38. % 将 32 位二进制字符转换为十进制整数, 得到消息长度
39. msg_len = bin2dec(char(len_bin + '0'));
40.
41. % 进行基本的安全检查, 防止解析出非法长度导致崩溃
42. if msg_len > (length(cH_vec) - start_idx - 32) || msg_len <= 0
43.     error('提取到的长度信息异常 (%d), 请检查图片是否正确或已被篡改!', msg_len);
44. end
45.
46. % 4. 根据提取出的长度信息, 提取秘密数据位
47. msg_bin = zeros(1, msg_len);
48. for i = 1:msg_len
49.     idx = start_idx + 32 + i;
50.     % 获取当前的 DWT 系数值
51.     c = cH_vec(idx);
52.     % 再次使用量化判决逻辑
53.     d0 = abs(c - round(c / delta) * delta);
54.     d1 = abs(c - (round((c - delta/2) / delta) * delta + delta/2));

```

```

55.     if d0 < d1
56.         msg_bin(i) = 0;
57.     else
58.         msg_bin(i) = 1;
59.     end
60. end
61.
62. % 5. 将二进制数据还原为原始字符串
63. msg_bin_mat = reshape(msg_bin, 8, []);
64. msg_bytes = bin2dec(char(msg_bin_mat + '0'));
65.
66. % 将字节流按 UTF-8 编码还原为原始字符
67. extracted_msg = native2unicode(uint8(msg_bytes)', 'utf-8');
68.
69. % 打印成功提取的提示和内容
70. disp('===== DWT 信息提取成功 =====');
71. % 输出提取出的隐藏秘密信息
72. disp(['提取出的隐藏信息为: ', extracted_msg]);
73.

```

五、实验结论及心得体会

本次实验中我们小组完成了随机 LSB 和 DWT 变换域隐写的任务。在实验中首先对 BMP 和 PNG 图像的格式内容进行了详细分析，并讨论了可能的隐藏位置和隐藏方法。接着小组成员使用 matlab 代码实现了 LSB 和 DWT 变换域两种隐写方法，成功实现对秘密的隐藏和提取，巩固了课上学习的隐写要求和隐写方法的基本原理。